High-Performance Rust for Mission-Critical Production Systems

> A deep-dive companion to the performance section of the README. Where the README is a scannable cheat-sheet, this doc explains the **why** behind each technique, the tradeoffs, and how to **model data and code so that performance falls out of the design** rather than being bolted on afterwards.
>
> Examples are framed around the kind of workload Rerun cares about: **high-throughput ingest of multimodal telemetry** (images, point clouds, tensors, time series), **columnar storage** (Apache Arrow), **zero-copy streaming**, and **GPU-driven visualization** of large datasets. The principles generalize to any low-latency system (trading, databases, game engines, media pipelines).

Table of contents

---

0. The performance mindset & methodology

Performance is a design property, not a patch. The order of leverage is almost always:

1. **Data layout & algorithms** (order-of-magnitude wins)
2. **Reducing work** — do less, do it once, do it lazily, batch it (large wins)
3. **Concurrency / parallelism** (linear-ish wins, bounded by contention & Amdahl)
4. **Micro-optimizations** — inlining, branchless, SIMD (last, and only where a profiler points)

A senior engineer spends most of their time in (1) and (2). Juniors reach for (4) first.

Measure the right thing: tails, not averages

In a streaming/serving system the **average latency is a vanity metric**. What users and downstream consumers feel is the **tail**: p99, p999, and max. A pipeline that is fast at p50 but stalls for 40 ms at p999 (allocator reclaim, lock convoy, page fault, GC-like drop storm) will drop frames or blow SLOs.

- Report **latency distributions** (histograms), not means. Use `hdrhistogram` for cheap, high-dynamic-range recording.
- **Throughput vs latency** are in tension (Little's Law: `concurrency = throughput × latency`). Batching raises throughput but can raise per-item latency — know which one you're optimizing.
- For a 1000-item request, if each sub-op has 1% chance of a 10 ms hiccup, the request almost always hits it. **Tail latency compounds with fan-out.**

Right tool for each question

Question	Tool
Which function is hot?	`perf` + `cargo-flamegraph`, `samply`
Is my microbench actually faster?	`criterion` (statistical, warmup, outlier detection)
Where do allocations come from?	`dhat`, `heaptrack`, `bytehound`
Am I cache-bound / branch-mispredicting?	`perf stat`, `cachegrind`, `perf record -e cache-misses`
What machine code did I get?	godbolt / Compiler Explorer, `cargo-asm`, `cargo-show-asm`
Is my async runtime stalling?	`tokio-console`
Did I introduce UB / data races?	`miri`, ThreadSanitizer, `loom`, `cargo-fuzz`

Benchmarking pitfalls (that make you optimize noise)

- **Dead-code elimination**: the optimizer deletes work whose result you ignore. Wrap inputs/outputs in `std::hint::black_box`.
- **Constant folding**: if inputs are literals, the compiler precomputes the answer. Feed runtime data.
- **Warmup & frequency scaling**: first iterations pay I-cache/branch-predictor cold-start and the CPU may not have ramped to turbo. Criterion handles warmup; pin frequency for stable numbers.
- **Measure release, matching prod CPU**: `--release`, and ideally the same `target-cpu` as production. A debug build tells you nothing (bounds checks on, no inlining, overflow checks on).
- **Benchmark the workload, not the benchmark**: microbenchmarks lie about cache behavior. Validate with an end-to-end load test that has realistic data sizes and access patterns.

---

1. Data modeling for performance — the biggest lever

> This is the section that matters most for a Rerun-style system, and the one that separates people who "know Rust" from people who build fast systems in Rust. **How you lay out data in memory determines how many cache lines the CPU has to touch, whether the loop auto-vectorizes, and how much you allocate.**

1.1 Array-of-Structs (AoS) → Struct-of-Arrays (SoA) / columnar

The single most important layout decision. Consider point-cloud points:

```
// AoS: fields interleaved. Iterating only `x` still drags y, z, intensity, _pad
// through cache. 32 bytes/point touched to read 4.
struct Point { x: f32, y: f32, z: f32, intensity: f32 }
struct Cloud { points: Vec<Point> }

// SoA / columnar: each field is its own contiguous buffer.
struct Cloud {
    x: Vec<f32>,
    y: Vec<f32>,
    z: Vec<f32>,
    intensity: Vec<f32>,
}
```

Why SoA wins for analytics/rendering/telemetry:

- **Cache efficiency**: a pass that needs only `intensity` streams one tight buffer — no wasted bytes, no padding pulled into L1.
- **Auto-vectorization**: `x: &[f32]` is a dense array the compiler can lower to SIMD (`+`, `*`, min/max) with no gather/scatter. Interleaved AoS usually can't vectorize.
- **Compression & encoding**: per-column data is homogeneous → RLE, delta, dictionary, bit-packing all work far better column-wise. This is exactly why **Apache Arrow (Rerun's data model) is columnar.**
- **Selective materialization**: queries touch only the columns they need. A "give me timestamps in [t0, t1]" query never reads the pixel buffers.

AoS still wins when you almost always touch *all* fields of *one* element at a time (e.g. a physics step that reads x,y,z,vx,vy,vz together). **Choose layout by access pattern**, and be willing to keep both a hot SoA and a cold side-table.

`SoA` in practice: crates like `soa_derive` generate the parallel-vec boilerplate; Arrow gives you columnar buffers with zero-copy slicing for free.

1.2 Data-Oriented Design (DOD) & ECS

Generalize SoA: **organize memory around how it's transformed, in bulk, not around "objects."** This is Data-Oriented Design (Mike Acton) and the core idea behind ECS (Entity-Component-System) engines and, in spirit, Rerun's entity/component/archetype model:

- Store components in contiguous columns keyed by entity.
- Systems iterate columns linearly → predictable, prefetchable, vectorizable.
- Entities are just IDs/indices, not owning pointers.

The mental flip: *"what does the transform want the memory to look like?"* beats *"what is the natural object model?"*.

1.3 Handles/indices instead of pointers (and `Arc` graphs)

Pointer-rich object graphs (`Rc<RefCell<Node>>`, `Box<dyn ..>` trees) are a triple performance tax: **pointer chasing** (each hop is a potential cache miss), **allocation per node**, and **lifetime/borrow pain**. Replace references with **indices into a dense arena**:

```
// Instead of Rc<RefCell<Node>> forming a graph:
struct Graph {
    nodes: Vec<Node>,          // contiguous, cache-friendly
    edges: Vec<(u32, u32)>,    // indices, not pointers — 4 bytes each
}
struct Node { /* ... */ }
type NodeId = u32;             // smaller than a 8-byte pointer
```

Benefits: contiguous storage (cache + SIMD), cheap `Copy` handles, trivial serialization, no borrow-checker fights, and you can `mem::swap`/reorder freely. For safe reuse without use-after-free/ABA, use **generational indices** (`slotmap`, `generational-arena`): each slot carries a generation counter so a stale handle is detected instead of aliasing a recycled slot.

1.4 Struct & enum layout: size, alignment, niches

Smaller data = more of it per cache line = fewer misses. Rust reorders fields by default to minimize padding, but you still control the total size.

- **`size_of`** / **`align_of`**: measure. A `struct` is padded up to its largest field's alignment. Ordering matters if you force `#[repr(C)]` (then *you* must order fields large→small to avoid padding).
- **Enums** cost `max(variant sizes) + discriminant`. A single fat variant bloats every value:

  ```
  enum Event {
      Tick(u8),
      Frame([u8; 4096]),   // makes EVERY Event 4 KB
  }
  // Fix: box the rare/large variant
  enum Event {
      Tick(u8),
      Frame(Box<[u8; 4096]>), // Event is now ~pointer-sized
  }
  ```

- **Niche optimization**: Rust packs discriminants into unused bit patterns. `Option<NonNull<T>>`, `Option<&T>`, `Option<NonZeroU32>`, `Option<Box<T>>` are the **same size as the inner value** — the null / zero pattern encodes `None` for free. Prefer `NonZeroU32` / `NonZero*` for ids and `NonNull` / `&T` so `Option<_>` stays free. Deep-nested `Option` / `Result` also collapse into one niche.
- **Shrink integers**: `usize` indices are 8 bytes; if your collection can't exceed 4 B elements, store `u32` indices (or `u16` / `u8`) and widen at use. Halving index size can halve a hot table's footprint.
- **`#[repr(packed)]`**: removes padding but creates **unaligned fields** — taking a reference to one is *undefined behavior*, and access is slower (or faults) on some targets. Use bit-packing or manual layout instead; reserve `packed` for wire/FFI structs and read through `read_unaligned`.

1.5 String & value interning

Repeated strings (entity paths like `/world/camera/points`, component names, symbols) are expensive to store, hash, and compare. **Intern** them into small integer symbols:

- One canonical copy per unique string; everything else holds a `u32` symbol.
- Equality and hashing become integer ops. Dedup shrinks memory dramatically.
- Crates: `string-interner`, `ustr`, `lasso`.

1.6 Zero-copy deserialization

The fastest parse is no parse. For high-rate ingest, avoid "read bytes → allocate owned structs":

- **Arrow / Arrow IPC / Flight**: columnar buffers are laid out so they can be read (and `mmap`'d, and sent over the wire) **without deserialization** — you point at the bytes. This is why Rerun can move data SDK→store→viewer cheaply.
- `bytes::Bytes`: a reference-counted, cheaply-cloneable, sliceable view into a shared buffer. Slicing is O(1) and allocation-free — ideal for parsing frames out of a network buffer without copying.
- `rkyv`: zero-copy deserialization framework — access your types directly in the byte buffer via an archived representation. Great for mmapped datasets and IPC.
- `Arc<[u8]>` / `Arc<[T]>`: share an immutable buffer across threads/consumers with no copy; drop the `capacity` word that `Vec` carries.
- `mmap` (`memmap2`): map a large read-only dataset; the OS page cache handles lazy paging and lets multiple processes share physical pages. Perfect for recordings/replays.

---

2. Memory & allocation

Allocation is a latency source, not just a throughput cost. `malloc` / `free` can take a global lock, touch a new page (page fault), or trigger arena reclamation — any of which shows up in your p999. In a mission-critical hot path, the goal is often **zero allocations in steady state.**

2.1 Allocate less, reuse more

- **Preallocate with capacity** when the size is known or bounded: `Vec::with_capacity`, `String::with_capacity`, `HashMap::with_capacity`. Avoids the geometric-growth realloc/copy chain.
- **Reuse buffers across iterations** instead of allocating fresh ones. Keep a scratch `Vec`, `clear()` it (keeps capacity), refill. For per-frame decode buffers this alone can remove most allocation.
- **`std::mem::take` / `mem::swap`**: move a value out behind `&mut self` leaving a cheap `Default`, avoiding a clone. Great for "process then reset" state machines and double-buffering.
- **Object pools** (`object-pool`, a `Vec` free-list) for expensive-to- create objects (large buffers, GPU staging).

2.2 Arena / bump allocation

For data with a common lifetime — a parse tree, a frame's transient objects, a request scope — a **bump allocator** allocates by incrementing a pointer and frees everything at once:

```
use bumpalo::Bump;
let arena = Bump::new();
let a = arena.alloc([0u8; 1024]);   // pointer bump, no per-object malloc
let b = arena.alloc(Node { /* .. */ });
// ... use a, b within the frame ...
// drop(arena): one deallocation frees everything. No per-node free calls.
```

Wins: near-zero allocation overhead, no per-object free traversal, excellent locality (objects allocated together sit together). `bumpalo`, `typed-arena`. Perfect for a renderer's per-frame allocations or a per-message decode scratchpad.

2.3 Small-buffer optimization (avoid the heap for the common small case)

When a collection is *usually* small, store it inline on the stack and spill to the heap only when it grows:

- `smallvec` / `arrayvec` / `tinyvec` for `Vec`-like.
- `compact_str` / `smartstring` for strings (inline up to ~24 bytes — covers most short strings with zero allocation).

2.4 Right-size the container

- **`Box<[T]>` / `Arc<[T]>` over `Vec<T>`** once it stops growing: drops the `capacity` field (one word saved per collection) and signals immutability. `Arc<[T]>` shares read-only data across threads with no copy.
- `Cow<'a, T>` for read-mostly data that is *occasionally* modified — borrow until you must own.
- **Choose the map**: `HashMap` for random access, `BTreeMap` for ordered/range scans, `Vec<(K,V)>` + binary search for small static maps (beats hashing for tiny N and is cache-friendly).

2.5 Custom global allocator

The system allocator is a reasonable default (Rust dropped bundled jemalloc as the default in Rust 1.32 in favor of the system allocator — smaller binaries, fewer platform issues). But for **allocation-heavy, multi-threaded** workloads, a modern allocator with per-thread caches often wins big by cutting cross-core contention:

```
#[global_allocator]
static GLOBAL: mimalloc::MiMalloc = mimalloc::MiMalloc;
// or tikv_jemallocator::Jemalloc
```

- `mimalloc`, `tikv-jemallocator`, `snmalloc`.
- Tradeoffs: larger binary, more RSS held in thread caches, another dependency. **Measure** — the win is workload-dependent (huge for many-small-alloc concurrent servers, negligible for alloc-light ones).
- jemalloc also exposes profiling and tunables (background threads, decay) useful in production.

2.6 Hidden allocations to watch for

`format!` / `to_string` in a loop, `.collect()` into a fresh `Vec` you immediately consume, `Box<dyn Fn>` closures, `String` where `&str` / `Cow` suffices, `.clone()` reflexes, trait objects that box. Profile with `dhat`; many "mysterious" allocations are one of these.

2.7 Bounded memory is a correctness property

The #1 way production Rust systems fall over is **OOM from unbounded growth** — an unbounded channel or queue behind a slow consumer, an ever-growing cache, retained history. **Bound everything**: bounded channels (apply backpressure), LRU/size-capped caches (`lru`, `moka`), ring buffers for fixed-window history, explicit eviction. Predictable memory beats "fast until it dies."

---

3. Low-latency & CPU-level performance (mechanical sympathy)

Once data layout is right, squeeze the CPU. **Mechanical sympathy** = writing code that works *with* the hardware (caches, pipelines, branch predictors, SIMD units) instead of against it.

3.1 The cache hierarchy is the performance model

Approximate latencies (see the full table):

Access	~Cycles	~Time
L1	4	~1 ns
L2	12	~4 ns
L3	40	~12 ns
Main memory (RAM)	200+	~60–100 ns

A main-memory miss costs **~100× an L1 hit**. This is why layout beats micro-ops: a cache-missy algorithm with fewer instructions loses to a cache-friendly one with more. **Cache line = 64 bytes** — memory moves in line-sized chunks, so touching one byte pulls 64.

Practical rules: iterate **contiguous** memory (`Vec` /slice, not linked list / `HashMap` -of-boxes); keep hot fields together and cold fields elsewhere (**hot/cold splitting**); prefer indices to pointers (§1.3); process in **cache-blocked** tiles for 2D/matrix data so a working set stays in L1/L2.

3.2 False sharing

Two threads writing to *different* variables that share a **cache line** force the line to ping-pong between cores (cache-coherence traffic) — invisible in the code, brutal in a profiler. Fix by padding hot per-thread/atomic data to its own line:

```
use crossbeam_utils::CachePadded;
struct Counters {
    a: CachePadded<AtomicU64>,  // each on its own 64-byte line
    b: CachePadded<AtomicU64>,
}
```

Classic case: an array of per-worker counters/atomics packed together. Pad them, or aggregate thread-local and merge at the end.

3.3 Branch prediction & branchless code

A mispredicted branch flushes the pipeline (~15–20 cycles). In tight loops over data:

- **Sort/partition data** so branches are predictable (all-taken then all-not-taken beats random).
- **Branchless** alternatives for hot conditionals: `select` -style arithmetic, `min` / `max` , `bool as u8` math, `slice::iter().filter(...)` . Let the compiler use conditional-moves.
- `#[cold]` on error/slow-path functions and `#[inline(never)]` to keep the hot path's I-cache clean; `std::hint::unlikely` / `likely` (stabilizing) or `#[cold]` calls to hint layout.

3.4 Bounds-check elimination

Rust inserts bounds checks on indexing; usually free-ish, but in hot numeric loops they block vectorization and add branches. Remove them **safely**:

- **Prefer iterators** (`for x in &slice`, `.iter().zip()`, `.windows()`, `.chunks_exact()`) — they carry the length invariant so the compiler proves indexing safe and drops checks.
- `chunks_exact(N)` gives the optimizer a fixed lane width → clean SIMD, no remainder branch in the body.
- **Hoist one assert**: `assert!(i < slice.len())` (or slicing to a known length up front) lets LLVM prove subsequent accesses safe and elide their checks.
- `get_unchecked` **is a last resort** — it's `unsafe`, must be provably in-bounds, and should be gated behind a comment stating the invariant and ideally checked in debug. See the bounds-check cookbook.

3.5 SIMD (vectorization)

One instruction on 4/8/16 lanes. Two routes:

- **Auto-vectorization (preferred)**: write loops the compiler can vectorize — dense `&[f32]`, no early returns/`?` in the body, no cross-iteration dependencies, `chunks_exact`. Verify on godbolt that you got `mulps`/`addps`/AVX ops. Enable the instructions with `target-cpu=native` or a specific `target-feature`.
- **Explicit SIMD** when the compiler won't cooperate: `std::simd` (portable, nightly), `wide`, `pulp` (portable with runtime feature dispatch). Great for point-cloud transforms, color conversion, dot products, min/max reductions over columns.

Caveat: `target-cpu=native` bakes in your build machine's ISA. In CI, build for the *production* CPU baseline, or ship multiple builds / use runtime feature detection (`is_x86_feature_detected!`).

3.6 Inlining & monomorphization

- Cross-crate inlining requires the callee to be **generic**, `#[inline]`, or **LTO** enabled — otherwise the optimizer can't see across the crate boundary.
- `#[inline(always)]` sparingly (it can bloat I-cache and *hurt*); `#[inline(never)]`/`#[cold]` to push cold code out of the hot path.
- **Monomorphization** (generics) gives static dispatch + inlining but costs compile time and binary size (code bloat → I-cache pressure). Balance with the "thin generic wrapper over a non-generic inner fn" trick to compile the body once.

3.7 Dynamic dispatch, judiciously

`&dyn Trait` / `Box<dyn Trait>` costs a vtable indirection and blocks inlining — avoid in inner loops. Alternatives:

- **Generics/monomorphization** for hot, few-types cases.
- `enum` **dispatch** (`enum_dispatch`) when the set of types is closed: a `match` the compiler can devirtualize and inline, with no allocation.
- Accept `dyn` freely on **cold paths** (plugin boundaries, config) where flexibility > nanoseconds, and to control code bloat.

3.8 Arithmetic micro-notes

- Rust floating-point is **not** `-ffast-math` by default (it preserves IEEE semantics for reproducibility — good for replay/determinism). Reassociation that would change results won't happen automatically.
- Integer **division/modulo** are slow; hoist reciprocals, use power-of-two masks (`& (n-1)`), or strength-reduce. `%` by a constant is optimized to multiply-shift.

- Overflow checks are on in debug, off (wrapping) in release; use `wrapping_*` / `checked_*` / `saturating_*` to be explicit rather than relying on profile.

---

4. Concurrency & parallelism

Concurrency is about **structure** (independent tasks); parallelism is about **execution** (simultaneous work). Rust's ownership model makes both far safer — use that to be aggressive. The performance ranking of strategies, best to worst: **shared-nothing → message passing → lock-free → fine-grained locks → coarse locks.**

4.1 Shared-nothing first: partition, don't lock

The fastest synchronization is none. **Partition data by key/shard** so each core owns a disjoint slice and never contends:

- Shard by entity path / stream id → per-shard state, no cross-talk on the hot path.
- Per-thread accumulators, merged once at the end (map-reduce). Zero contention during the work.
- This is also how **thread-per-core** runtimes get their speed (see §4.6).

4.2 Data parallelism with Rayon

For CPU-bound bulk work over a collection, `rayon` turns `.iter()` into `.par_iter()` with a work-stealing pool:

```
use rayon::prelude::*;
let sum: f64 = points.par_iter().map(|p| p.intensity as f64).sum();
points.par_chunks_mut(4096).for_each(|chunk| transform(chunk));
```

Great scaling when work per item is non-trivial and N is large. **Overhead is real for tiny work** — don't `par_iter` a 50-element vector of cheap ops; the scheduling costs more than it saves. Use `with_min_len` to control granularity.

4.3 When you must share: choose the primitive

- **`parking_lot::Mutex/RwLock`** over `std` for hot locks: smaller (1 byte), faster uncontended path, adaptive spin-then-park (mostly stays in user space), no poisoning overhead. `std::sync` gained speedups too, but `parking_lot` is still a solid default for contended short critical sections.
- **`RwLock`** only when reads *heavily* dominate and the critical section is non-trivial; otherwise a `Mutex` is often faster (RwLock bookkeeping isn't free, and writers can starve).
- **Keep critical sections tiny**: clone/copy the needed data out under the lock, release, then do the heavy work. Never hold a lock across `.await` or I/O.

4.4 Reduce contention

- **Shard the lock**: `[Mutex<Shard>; N]` indexed by `hash(key) % N`, or `dashmap` (a sharded concurrent `HashMap`). Turns one hot lock into N cool ones.
- **Read-mostly config/state**: `arc-swap` for RCU-style atomic pointer swap — readers get a lock-free `Arc` snapshot, writers publish a new `Arc`. Ideal for hot-reloaded settings, routing tables, current-frame state.
- **Seqlock** for a single writer + many readers of small POD (a pose/transform, a counter): readers spin on a version and retry if it changed — no writer starvation, no reader locks.

4.5 Lock-free & atomics (know the memory model)

Atomics let you build lock-free structures, but **memory ordering is the whole game**:

Ordering	Meaning	Use
`Relaxed`	Atomicity only, no ordering	counters/stats where order doesn't matter
`Acquire` (load)	No later op moves before it	pair with a `Release` store to *see* published writes
`Release` (store)	No earlier op moves after it	publish data before flipping a flag
`AcqRel`	Both, for read-modify-write	`fetch_add` / `compare_exchange` that both consumes & publishes
`SeqCst`	Single global total order	when you truly need cross-variable global ordering (most expensive)

- The **Acquire/Release pair** is the workhorse: writer stores data (Relaxed), then `Release`-stores a ready flag; reader `Acquire`-loads the flag, then reads the data (Relaxed) and is guaranteed to see it.
- **`compare_exchange_weak` in loops**: it may fail spuriously but compiles to cheaper LL/SC on ARM and avoids a nested loop — always the loop form; use the non-weak `compare_exchange` for one-shot.
- **Beware ABA**: a value can change A→B→A between your read and CAS. Use generational tags or **epoch-based reclamation** (`crossbeam-epoch`) to free nodes safely.
- **Don't hand-roll** lock-free queues/stacks in production unless you must — reach for `crossbeam` (epoch GC, deques, `SegQueue`, `ArrayQueue`) and validate any custom `unsafe` concurrency with `loom`, which explores interleavings exhaustively.

4.6 Channels & runtime architecture

- **Channels**: `crossbeam-channel` / `flume` (sync), `tokio::sync::mpsc` (async). **Bounded by default** for backpressure (unbounded = latent OOM, §2.7). For the lowest-latency single-producer/single-consumer, a lock-free ring buffer (`rtrb`) beats a general channel.
- **Work-stealing (`tokio`, `rayon`)**: balances uneven load automatically; tasks may migrate across cores (cache cost, needs `Send`).
- **Thread-per-core (`glommio`, `monoio`)**: pin one executor per core, shard data per core, share nothing → no cross-core sync, great tail latency and cache locality for high-throughput ingest/serving. Tradeoff: you must partition work evenly. This is the architecture to reach for in a "millions of messages/sec, tight p999" ingest path.
- **Pin threads** (`core_affinity`) and be **NUMA-aware** (allocate memory on the socket that touches it) for the last slice of latency on big machines. Avoid **oversubscription** (more busy threads than cores → scheduler thrash).

---

5. Async & streaming pipelines

Async is for **I/O concurrency** (thousands of sockets), not a speed button. Understand its cost so you apply it where it pays.

5.1 How it actually runs: futures, poll, wakers, and tasks

You can't reason about async performance (or cancel-safety, or "why is my future huge / `!Send`") without the mechanical model. The whole thing fits in one trait:

```
pub trait Future {
    type Output;
    // The executor drives a future by calling poll until it returns Ready.
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}
enum Poll<T> { Ready(T), Pending }
```

Futures are lazy and do nothing until polled. An `async fn` doesn't run when called — it *returns* a future. Nothing happens until an executor polls it (or you `.await` it, which polls it from a parent).

`async` / `await` lowers to a state machine. The compiler turns each `async fn` into an enum that implements `Future`; every `.await` is a **suspension point** = one state, and every local that must stay alive across an await is stored **inside** that enum:

```
async fn handle(sock: &TcpStream, buf: &mut [u8]) -> usize {
    let n = sock.read(buf).await;   // suspension point
    process(n)
}
// conceptually lowers to:
enum Handle<'a> {
    Start   { sock: &'a TcpStream, buf: &'a mut [u8] },
    Reading { read: ReadFuture<'a> },   // in-progress leaf future is kept here
    Done,
}
// impl Future for Handle { fn poll(..) { match *self { /* advance the state */ } } }
```

No heap allocation for the future itself — it's a plain value sized on the stack (unless you `Box::pin` it). Its **size = the largest set of locals simultaneously live across awaits.**

The life of a wakeup (why it doesn't busy-wait):

1. The **executor** pops a ready **task** off a worker's run queue and calls `poll` on its root future.
2. `poll` descends the future tree until it hits a **leaf** — a socket read, a timer, a channel `recv`. If the resource isn't ready, the leaf stashes the task's `Waker` (from `cx`) with the **reactor** and returns `Poll::Pending`; every parent propagates `Pending` up.
3. The executor **parks** the task (removes it from the run queue). The worker thread immediately runs *other* ready tasks — no thread is blocked, no spinning.
4. The **reactor / driver** (Tokio uses `mio` → epoll/kqueue/IOCP for I/O, plus a timer wheel for time) sleeps in **one** syscall on *all* registered sources. When the OS reports readiness, it calls the stored `waker.wake()`.
5. `wake()` pushes the task back onto a run queue; the executor **re-polls** it, the state machine `match`es straight to where it suspended, and resumes.

This is **cooperative, not preemptive**: progress happens only at `.await` points, and a future must return `Pending` for the scheduler to run anything else on that thread.

Task vs future. A *future* is any resumable computation; futures compose into a tree via `.await` / `join!` / `select!`, and a parent polls its children directly. A **task** is a *top-level* future you hand to the executor (`tokio::spawn`) plus its bookkeeping (state, `Waker`, `JoinHandle`). **Only the task root sits on the run queue and is scheduled** — everything it `.await`s is polled inline by the task, on one stack frame. That's why a task is the unit of concurrency but not every future is.

Executor vs reactor. Tokio cleanly splits the two: the **scheduler** owns worker threads + run queues and does work-stealing; the **reactor** owns the OS readiness/timer machinery and turns events into `wake()` calls. They meet only through the `Waker`. (Thread-per-core runtimes like `glommio` / `monoio` fuse a scheduler + reactor per core and share nothing — §4.6.)

Pinning exists because the state machine is self-referential. If a local borrows another local across an await, the future holds a pointer *into itself*; moving it would dangle that pointer. `Pin<&mut F>` is the guarantee "this future won't move again," which is why `poll` takes `Pin<&mut Self>`, why you `tokio::pin!` / `Box::pin` before polling in place, and why **async recursion needs boxing** (an un-boxed state machine that contains itself has infinite size).

Cancellation is just `drop`. Because a future keeps *all* its in-progress state in the state machine and does nothing unless polled, **dropping it cancels it** — its fields (including any partially filled buffers) are dropped, no special teardown. This is precisely why `select!` "cancels" the losing branches (it drops them) and why **cancel-safety** (§5.7) is the question: *does dropping this future mid-flight lose data or corrupt shared state?*

Why all of this shapes performance:

- **Future size matters.** Deep async call trees produce large state machines; large futures are costly to move and blow the cache. Boxing a rarely-taken branch (or a giant future) shrinks the common path — the async analogue of §1.4's "box the fat enum variant."
- **No per-task stack.** Unlike threads (~MBs of stack each), a task is just its state machine on the heap of whoever holds it → millions of concurrent tasks are feasible. The cost moved from memory to poll/resume overhead (a `match` + waker traffic per suspension).
- **`Send` is a scheduling constraint, not pedantry.** A multi-thread executor may resume a task on a *different* worker after an await, so everything held **across** an await must be `Send` — this is the real reason `Rc` or a `std::MutexGuard` held across `.await` makes a future un-spawnable (§5.3).
- **Wakeups aren't free** (atomics + queueing); spurious wakeups waste a full poll. And a task that never returns `Pending` (a tight CPU loop with no awaits) starves its worker — Tokio's per-task **budget** forces periodic yields at await points, but a loop with *no* await won't yield (→ §5.3).

5.2 When async helps vs hurts

- **Helps**: many concurrent I/O-bound tasks (network fetches from exchanges/sensors/services, thousands of connections) — one OS thread multiplexes them cheaply.
- **Hurts**: CPU-bound or latency-critical inner loops. `async fn`s compile to **state machines** polled by a scheduler; you pay poll/wake overhead, less predictable memory access, and larger, harder-to-inline futures. For a tight compute kernel this is pure overhead.
- **Hybrid (the production sweet spot)**: async at the **I/O boundary** (ingest, network, disk), synchronous (and possibly rayon-parallel) code for the **decision/compute core** where latency must be predictable. Bridge them with channels. This keeps determinism where it matters and concurrency where it pays.

5.3 Never block the executor

A blocking call (heavy CPU, `std::fs`, a `Mutex` held long, a sync DB call) on an async worker stalls *every* task on that thread. Offload:

- `tokio::task::spawn_blocking` for blocking syscalls/CPU.
- A dedicated rayon pool or thread pool for heavy compute, results returned via channel.
- Never hold a `std::sync::Mutex` across `.await` (use `tokio::sync::Mutex` only when you must, and keep it short). Prefer "copy out, drop guard, await."

5.4 Backpressure is mandatory in streams

A `Stream` producer faster than its consumer will buffer without bound → OOM. Build backpressure in:

- **Bounded channels**: `send().await` suspends the producer when full — natural backpressure that propagates upstream to the source (e.g. slow the sensor read / pause the socket).
- Avoid unbounded `buffer_unordered` / `FuturesUnordered` growth; cap in-flight concurrency.
- Design for **load shedding** at the edge: when overloaded, drop or downsample rather than queue forever (a viewer can drop frames; it can't recover from OOM).

5.5 Zero-copy & syscall-efficient I/O

- **`bytes::Bytes`** through the whole pipeline: parse frames as O(1) reference-counted slices of the receive buffer — no copy from socket → parser → store.
- **Batch syscalls**: `BufWriter`, vectored I/O (`writev` / `write_vectored`) to coalesce many small buffers into one syscall; batch messages per wakeup. Syscalls are ~hundreds of ns to µs each — amortize them.
- **`io_uring`** (`tokio-uring`, `glommio`, `monoio`) for high-IOPS disk/network: batched submission/completion, fewer syscalls, optional zero-copy — a real win for a storage/ingest layer.

5.6 Wire format & serialization

The serializer is often the hidden bottleneck in a streaming system:

- **Columnar/zero-copy on the wire**: Arrow IPC / Arrow Flight (gRPC) move columnar batches with minimal transform — Rerun's SDK→viewer transport is in this family. Reading can be near zero-copy.
- **Avoid JSON/text in hot paths**. For compact binary use `bincode`, `postcard` (no-std friendly), protobuf (`prost`), or `rkyv` / `capnp` for zero-copy access.
- **Batch small messages** into larger frames (fewer headers, better compression, fewer syscalls) — trading a little latency for a lot of throughput, tuned to your SLO.

5.7 Async correctness that affects latency

- **Cancellation safety**: dropping a future must not corrupt state; know which combinators are cancel-safe (`tokio::select!` drops the losers).
- **Fairness**: a hot `select!` branch can starve others; use `tokio::task::yield_now`/budgeting for long loops. Diagnose stalls with **`tokio-console`**.
- **Task granularity**: millions of tiny tasks add scheduler overhead; batch work per task.

5.8 Tokio idioms & recipes (production patterns)

Concrete patterns that show up in every serious Tokio codebase. Each is chosen for a **performance/reliability** reason, not just style.

Choosing & building the runtime

`#[tokio::main]` is fine for apps, but build the runtime explicitly when you want to **size the pool**, name threads for observability, or **isolate latency-sensitive I/O from heavy work**:

```
let rt = tokio::runtime::Builder::new_multi_thread()
    .worker_threads(4)          // don't oversubscribe cores; leave room for compute pools
    .thread_name("io-worker")   // shows up in perf/tokio-console
    .enable_all()               // I/O + time drivers
    .build()?;
rt.block_on(async { /* ... */ });
```

- `current_thread` (`#[tokio::main(flavor = "current_thread")]`) for CLIs, tests, and thread-per-core designs: a single-threaded event loop, no work-stealing overhead, and futures needn't be `Send`.
- **Two runtimes** is a legitimate pattern: a small dedicated runtime pinned to the **ingest socket** (tight tail latency) and a separate pool for background/compute tasks, so a burst of heavy work can't delay I/O polling. Bridge them with channels.

The actor pattern — own state in a task, not behind a lock

The most important Tokio idiom for shared mutable state. Instead of `Arc<Mutex<State>>` (lock contention, the across-`.await` footgun — §5.3), give the state to **one task** and talk to it via an `mpsc` channel; carry a `oneshot` sender for the reply. Access is serialized, lock-free, and never blocks the executor:

```
enum Command {
    Get { key: String, reply: oneshot::Sender<Option<Bytes>> },
    Set { key: String, val: Bytes },
}

async fn actor(mut rx: mpsc::Receiver<Command>) {
    let mut state: HashMap<String, Bytes> = HashMap::new();
    while let Some(cmd) = rx.recv().await {          // recv() is cancel-safe
        match cmd {
            Command::Get { key, reply } => { let _ = reply.send(state.get(&key).cloned()); }
            Command::Set { key, val }   => { state.insert(key, val); }
        }
    }
}

// A cheap, Clone-able handle hides the channel from callers.
#[derive(Clone)]
struct Handle { tx: mpsc::Sender<Command> }
impl Handle {
    async fn get(&self, key: String) -> Option<Bytes> {
        let (reply, rx) = oneshot::channel();
        self.tx.send(Command::Get { key, reply }).await.ok()?;  // bounded send = backpressure
        rx.await.ok().flatten()
    }
}
```

Bound the `mpsc` so a slow actor pushes back on callers (§2.7). This scales to sharded actors (one per entity/stream) for a shared-nothing pipeline (§4.1).

Graceful shutdown & structured concurrency

Don't `abort()` tasks and lose in-flight work. Broadcast a **cancellation signal**, then **await the outstanding tasks** to drain. `CancellationToken` and `TaskTracker` live in `tokio-util`:

```rust
use tokio_util::{sync::CancellationToken, task::TaskTracker};

let token = CancellationToken::new();
let tracker = TaskTracker::new();

for _ in 0..workers {
    let token = token.clone();
    tracker.spawn(async move {
        loop {
            tokio::select! {
                biased;
                _ = token.cancelled() => break,        // shutdown wins deterministically
                job = next_job()       => handle(job).await,
            }
        }
    });
}

tokio::signal::ctrl_c().await?;   // or a SIGTERM stream
token.cancel();                    // tell every task to wind down
tracker.close();
tracker.wait().await;              // block until all spawned tasks finish (graceful drain)
```

`CancellationToken` also has `child_token()` for hierarchical shutdown (cancel a subtree without touching the rest) and `drop_guard()` to auto-cancel on scope exit.

Bounded fan-out (JoinSet + Semaphore)

Run many tasks with a hard cap on concurrency — the canonical fix for "fetch 10 000 URLs without opening 10 000 sockets." The `Semaphore` provides backpressure; `JoinSet` collects results and surfaces panics:

```rust
use std::sync::Arc;
use tokio::{sync::Semaphore, task::JoinSet};

let sem = Arc::new(Semaphore::new(16));   // at most 16 in flight
let mut set = JoinSet::new();
for url in urls {
    let permit = sem.clone().acquire_owned().await.unwrap();  // suspends here when saturated
    set.spawn(async move {
        let _permit = permit;             // released on task completion (drop)
        fetch(url).await
    });
}
while let Some(res) = set.join_next().await {
    match res {
        Ok(body)                => { /* use it */ }
        Err(e) if e.is_panic()  => { /* a task panicked — isolated, not fatal */ }
        Err(_)                  => { /* task was cancelled/aborted */ }
    }
}
```

`futures::stream::iter(..).map(..).buffer_unordered(16)` is a more compact equivalent when you have a stream and don't need per-task panic handling.

Timeouts, backoff, and periodic ticks

```rust
use tokio::time::{timeout, interval, Duration, MissedTickBehavior};

// Bound any operation. Note the *nested* Result: outer = timeout, inner = the op.
match timeout(Duration::from_millis(200), fetch()).await {
    Ok(Ok(v))   => { /* success */ }
    Ok(Err(e))  => { /* fetch failed */ }
    Err(_elapsed) => { /* timed out — the fetch future is dropped/cancelled here */ }
}

// Fixed-rate ticker that does NOT drift and won't burst-fire after a stall:
let mut tick = interval(Duration::from_millis(10));
tick.set_missed_tick_behavior(MissedTickBehavior::Delay);  // skip make-up ticks
loop {
    tick.tick().await;          // prefer this over sleep-in-a-loop (which accumulates drift)
    do_periodic_work().await;
}
```

Exponential backoff with a cap (add **jitter** in production to avoid thundering herds; crates `backon` / `tokio-retry` handle jitter + deadlines):

```rust
let mut delay = Duration::from_millis(10);
let value = loop {
    match do_request().await {
        Ok(v) => break v,
        Err(_) if delay < Duration::from_secs(1) => {
            tokio::time::sleep(delay).await;
            delay *= 2;
        }
        Err(e) => return Err(e),
    }
};
```

`select!` in a loop — cancel-safety without losing data

In `loop { select! { .. } }`, each branch's future is **recreated every iteration** and the losers are **dropped**. That's harmless for cancel-safe ops (`recv()`, `token.cancelled()`, `tick()`), but discards partial progress for **non-cancel-safe** ones (`read_exact`, `write_all` — they may have buffered bytes, §5.7). Fix: create the non-cancel-safe future **once**, `pin!` it, and poll `&mut` it so a losing round keeps its state:

```rust
let read = conn.read_exact(&mut buf);
tokio::pin!(read);
loop {
    tokio::select! {
        biased;                                  // deterministic order: check shutdown first
        _    = token.cancelled() => break,
        res = &mut read         => { handle(res)?; break; }  // partial reads preserved across rounds
        _    = tick.tick()       => heartbeat().await,
    }
}
```

`biased;` disables the default random branch polling — cheaper and lets you prioritize shutdown, at the cost of fairness (a always-ready top branch can starve lower ones, so order deliberately).

Bridging to CPU-bound work

Never run a heavy sync computation on an async worker (§5.3). Offload to a blocking thread or a rayon pool and get the result back over a `oneshot`, keeping the async task free to make progress:

```
let (tx, rx) = oneshot::channel();
rayon::spawn(move || {
    let result = expensive_transform(&data);   // runs on the rayon pool, off the async runtime
    let _ = tx.send(result);
});
let result = rx.await?;                          // async worker stays available meanwhile
```

Use `tokio::task::spawn_blocking` for blocking *syscalls* (file I/O, a sync DB driver); use a **rayon pool** for CPU-parallel compute — `spawn_blocking` threads are meant to park on I/O, not saturate cores.

Shared state, when you must

If the actor pattern is overkill, share directly — but pick the right primitive:

- `Arc<std::sync::Mutex<T>>` for short, synchronous critical sections that **never** span an `.await` (cheaper than `tokio::sync::Mutex`; §5.3). "Lock → copy out → drop guard → await."
- `tokio::sync::watch` for broadcast latest-value state (config, current frame) — receivers see only the newest value, perfect for state that supersedes.
- `arc-swap` for read-mostly snapshots readers grab lock-free (§4.4).

Pitfalls & instrumentation

- **Dropping a `JoinHandle` detaches the task** — it keeps running unsupervised. If you need its result or completion, await it, or manage it in a `JoinSet` / `TaskTracker`.
- **A panic in a task doesn't crash the runtime**; it surfaces as `Err(JoinError)` with `is_panic()`. Silently detached tasks swallow panics — another reason to track handles.
- **A tight CPU loop with no `.await` never yields.** Tokio's cooperative budget only yields *at* await points; insert `tokio::task::yield_now().await` in long loops or offload the work.
- **Diagnose stalls with `tokio-console`** (long poll times = a blocking call on the runtime); name tasks via `tokio::task::Builder::new().name(..)` so they're identifiable.

---

6. Mission-critical production concerns

Where "fast on my laptop" becomes "reliably fast in production." **Predictability is a feature.**

6.1 Tail latency is the product

Enumerate and design out the p99/p999 spike sources:

Spike source	Mitigation
Allocator stalls / reclaim	Preallocate, pools, arenas; steady-state zero-alloc; tuned allocator
Lock convoys / contention	Shard, lock-free, shorter critical sections, per-core state
Page faults	Pre-touch/ `mlock` critical buffers; huge pages; warm the working set
Syscalls	Batch/ `io_uring`; buffer writes
Drop storms	Free large structures off the hot path (send to a cleanup thread)
Background GC-like work	Bound and schedule it; incremental compaction
Thread migration / cold caches	Pin threads, thread-per-core, NUMA-local allocation

Steady-state, allocation-free hot loops are the recipe for flat tails: allocate everything up front, reuse buffers, and let no operation do unbounded work per iteration.

6.2 Panic policy & FFI

- `panic = "abort"` in production hot systems removes unwinding machinery (smaller binary, no landing-pad codegen, marginally faster), and unwinding **across an FFI boundary is UB** — relevant for Rerun's C++/Python bindings. Catch/handle at boundaries with `catch_unwind` if you need to keep a service alive, but the default posture for a data-plane binary is abort-and-restart under a supervisor.
- Keep panics off the hot path; validate inputs at the edge (parse-don't-validate, §1) so inner loops can't panic.

6.3 Determinism & reproducibility (robotics/replay)

Rerun records and **replays** — determinism is a first-class concern, and it also aids debugging and testing:

- **Don't depend on `HashMap` iteration order** (randomized per-run). Use `IndexMap` / `BTreeMap`, sort before iterating, or a fixed-seed hasher where order matters.
- **Seed RNGs** explicitly; thread a seed through, don't use thread-random in logic that must replay.
- **FP determinism**: Rust's non-fast-math default helps; still beware cross-platform FP and parallel reduction order (`par_iter().sum()` can reassociate — use a deterministic reduction when bit-exactness matters).
- **Stable ordering** of concurrent outputs when the consumer expects order (tag with sequence numbers, reorder at the sink).

6.4 Low-overhead observability

You can't fix p999 you can't see — but naive instrumentation *creates* p999:

- **Atomic counters + histograms** (`hdrhistogram`) instead of logging in inner loops (logging/formatting in a hot loop is a classic self-inflicted stall — see the README note).
- **Sampling** profilers in prod (`perf`, `samply`, continuous profiling) — near-zero overhead vs instrumenting every call.

- `tracing` with spans kept off the hot path and cheap subscribers; sample or rate-limit. Prefer structured counters over string logs for high-frequency events.

6.5 Validating `unsafe` and concurrency

Performance work leans on `unsafe` (get_unchecked, custom SIMD, lock-free). Earn it back with tooling:

- `miri` for UB in `unsafe` (out-of-bounds, uninit, aliasing, invalid values).
- **ThreadSanitizer / `loom`** for data races and concurrency interleavings (loom exhaustively explores orderings for a lock-free structure under test).
- `cargo-fuzz` **/ property tests** for parsers and codecs on untrusted input.
- **Document every `unsafe` block's invariant** and check it in debug (`debug_assert!`). Unsafe without a written contract is a future incident.

6.6 Build & deploy for speed

- Release profile for max runtime perf (slower compile, worth it for the data plane):

```
[profile.release]
lto = "fat"          # whole-program optimization across crates (~up to 20%)
codegen-units = 1    # one unit → best optimization, no parallel-codegen penalty to perf
panic = "abort"      # drop unwinding
# opt-level = 3 is default in release
```

- **Match the production CPU** (`target-cpu`) so LTO/SIMD use the right ISA — but keep CI's baseline compatible with the oldest deployed hardware, or ship per-arch builds.
- **PGO** (profile-guided optimization) and **BOLT** (post-link binary layout optimizer) for the last 5–15% on a stable, representative workload — worthwhile for a shipped viewer/server binary.
- Beware: `debug_assert!` and overflow checks are *off* in release — don't rely on them for safety in production; keep real checks explicit.

---

7. Synthesis: designing a Rerun-like ingest → store → render pipeline

Putting it together for a high-throughput multimodal telemetry system — the choices and *why*:

Ingest (SDK → transport):

- Accept data in **columnar batches** (Arrow), not per-row — amortizes overhead and keeps everything vectorizable/compressible downstream (§1.1).
- **Zero-copy from wire**: parse frames as `bytes::Bytes` slices; move Arrow buffers without deserializing (§1.6, §5.5). Transport over gRPC/Arrow Flight.
- **Backpressure** end-to-end: bounded channels so a slow store slows the producer instead of OOMing (§2.7, §5.4). **Thread-per-core** ingest, sharded by entity path, for flat p999 (§4.6).

Store (chunked columnar):

- **Columnar chunk store** indexed by entity + timeline: each component is a contiguous column; queries ("components on `/world/points` in `[t0,t1]`") touch only relevant columns and time ranges (§1.1).
- `Arc<[_]>` **/ Arrow buffers** shared read-only across query threads and the renderer — no copy (§1.6, §2.4). `mmap` cold recordings for replay so the OS pages them lazily (§1.6).
- **Bounded memory**: cap in-memory history; evict/compact to disk; ring-buffer live windows (§2.7).
- **Generational-index handles** for entities/instances instead of pointer graphs (§1.3).

Query & compute:

- **SoA/SIMD** over columns for transforms, filtering, reductions (§1.1, §3.5). **Rayon** across chunks for bulk queries; shard to avoid contention (§4.1–4.2).
- **Arena per query/frame** for transient results, freed in one shot (§2.2). Steady-state buffer reuse for zero-alloc hot paths (§2.1).

Render (viewer on wgpu):

- **Zero-copy to the GPU**: upload columnar buffers directly; keep CPU-side data in the layout the GPU wants (interleaved vertex buffers or SoA per attribute) to avoid a repack.
- **Per-frame bump arena** for transient render data; **object pools** for staging buffers (§2.2–2.1).
- **Async I/O boundary, synchronous render core** — predictable frame timing, no executor overhead in the hot loop (§5.2). Drop/downsample frames under load rather than stall (§5.4).

The through-line: **columnar + zero-copy + bounded + sharded**, with allocation pushed to arenas/pools and kept out of steady state. Performance is a consequence of the data model, not a later optimization pass.

---

8. Appendix

Latency numbers

Rough orders of magnitude every systems engineer should internalize (modern server, ~ns):

Operation	Latency
L1 cache reference	~1 ns
Branch mispredict	~3–5 ns
L2 cache reference	~4 ns
Mutex lock/unlock (uncontended)	~15–25 ns
L3 cache reference	~12 ns
Main memory reference	~60–100 ns
Syscall (light)	~100 ns – 1 µs
Context switch	~1–5 µs
SSD random read	~15–150 µs
Datacenter round trip	~0.5 ms
Disk (HDD) seek	~5–10 ms

The takeaways: a **RAM miss ≈ 100 L1 hits**; a **context switch ≈ 1000 L1 hits**; an **SSD read ≈ a context switch × 10s**. Design to stay high in this table.

Profiling & analysis tools

Tool	Purpose
`criterion`	Statistical microbenchmarks (warmup, outliers, regression)
`cargo-flamegraph` / `perf` / `samply`	Sampling CPU profiler → flamegraphs
`dhat` / `heaptrack` / `bytehound`	Heap allocation profiling
`perf stat` / `cachegrind`	Cache misses, branch mispredicts, IPC
`cargo-show-asm` / godbolt	Inspect generated assembly (did it vectorize/inline?)
`tokio-console`	Async task stalls, poll times, resource waits
`miri`	UB detection in `unsafe`
`loom`	Exhaustive concurrency interleaving tests
`cargo-fuzz`	Fuzzing parsers/codecs
`cargo bloat` / `cargo-llvm-lines`	Binary size & codegen hotspots

Performance-relevant crates

Need	Crate
Small-buffer opt	`smallvec`, `arrayvec`, `tinyvec`, `compact_str`, `smartstring`
Arena / bump	`bumpalo`, `typed-arena`
Generational handles	`slotmap`, `generational-arena`
Zero-copy buffers	`bytes`, `arrow`, `rkyv`, `memmap2`
Faster hashing	`ahash`, `rustc-hash` (`FxHashMap`), `foldhash`, `nohash-hasher`
Concurrency	`crossbeam`, `parking_lot`, `dashmap`, `arc-swap`, `rtrb`
Data parallelism	`rayon`
Async runtime & utils	`tokio`, `tokio-util` (`CancellationToken`, `TaskTracker`), `futures`, `backon` / `tokio-retry`
Custom allocator	`mimalloc`, `tikv-jemallocator`, `snmalloc-rs`
SIMD	`std::simd`, `wide`, `pulp`
Cache padding	`crossbeam-utils::CachePadded`
Histograms	`hdrhistogram`
Interning	`string-interner`, `ustr`, `lasso`
Bounded caches	`lru`, `moka`

A performance investigation checklist

1. **Reproduce & measure** with a realistic workload; capture p50/p99/p999, not just mean.
2. **Profile before touching code** — flamegraph for CPU, `dhat` for allocations. Find *the* bottleneck.
3. **Is it the data layout?** (cache misses, pointer chasing, AoS) → fix layout first (§1).
4. **Is it allocation?** (allocs in hot loop) → preallocate/reuse/arena (§2).
5. **Is it contention?** (lock time, false sharing) → shard/lock-free/pad (§4).
6. **Is it the algorithm?** (big-O, redundant work) → reduce work before optimizing constants.
7. **Only then micro-optimize** (branchless, SIMD, inlining) — and **re-measure** each change; keep it if the benchmark (and the tail) actually improved.
8. **Guard the win**: add a regression benchmark and validate any new `unsafe` with miri/loom.

Further reading

- The Rust Performance Book — Nethercote
- cheats.rs — dense language reference
- Rust Atomics and Locks — Mara Bos (the definitive concurrency/atomics text)
- Data-Oriented Design — Fabian (mirror of Acton's ideas)
- Agner Fog's optimization manuals — CPU microarchitecture
- Bounds-check cookbook
- Rerun's architecture blog — columnar store, Arrow, and the viewer in practice